



KILID

Secure Desktop Messenger

End-to-End Encrypted · Open Source · Privacy First

E2E Encryption

MQTT Transport

Groups & Voice

Built-in Games

KILID is a modern, end-to-end encrypted desktop messenger built for privacy. Messages are encrypted on your device before they ever leave it. The application uses public MQTT brokers for transport while keeping message content unreadable to servers. Identity is device-bound, recovery is offline-friendly, and the UI follows a refined dark glass aesthetic designed for long daily use.

Product Introduction · Version 1.0.0 · 2026

<https://kilid.app>

github.com/armanghobadi/kilid

What is KILID?

KILID Messenger is a privacy-focused desktop application for secure communication. It combines strong cryptography (ECDH key exchange, AES-GCM, PBKDF2), a lightweight MQTT transport layer, and a polished PyQt6 interface. There is no central message server that can read your content — only the devices you choose can decrypt conversations.

Core Capabilities

End-to-End Encryption

ECDH (SECP384R1) key agreement, AES-GCM message encryption, and optional key rotation / ratcheting for session security.

Contacts & Groups

Private 1:1 chats and encrypted group rooms with shared group keys, invite codes, PIN-protected join, and admin roles.

Voice & Media

Record and send voice messages (up to 60s), files up to 10 MB with chunked transfer, images, and system notifications.

Account Recovery

32-digit offline Safety Number, encrypted .kilidbak backup files, and online device-to-device transfer over MQTT.

Built-in Games

Play Backgammon or Chess vs bot, practice mode, or encrypted online matches with contacts via the same secure channel.

Flexible Transport

Works with public or private MQTT brokers (HiveMQ, Mosquitto, EMQX). TLS preferred; broker config is per-account.

Design Principles

Privacy by Default

No message content on servers

Local-First

Keys & history stay on device

Transparent Crypto

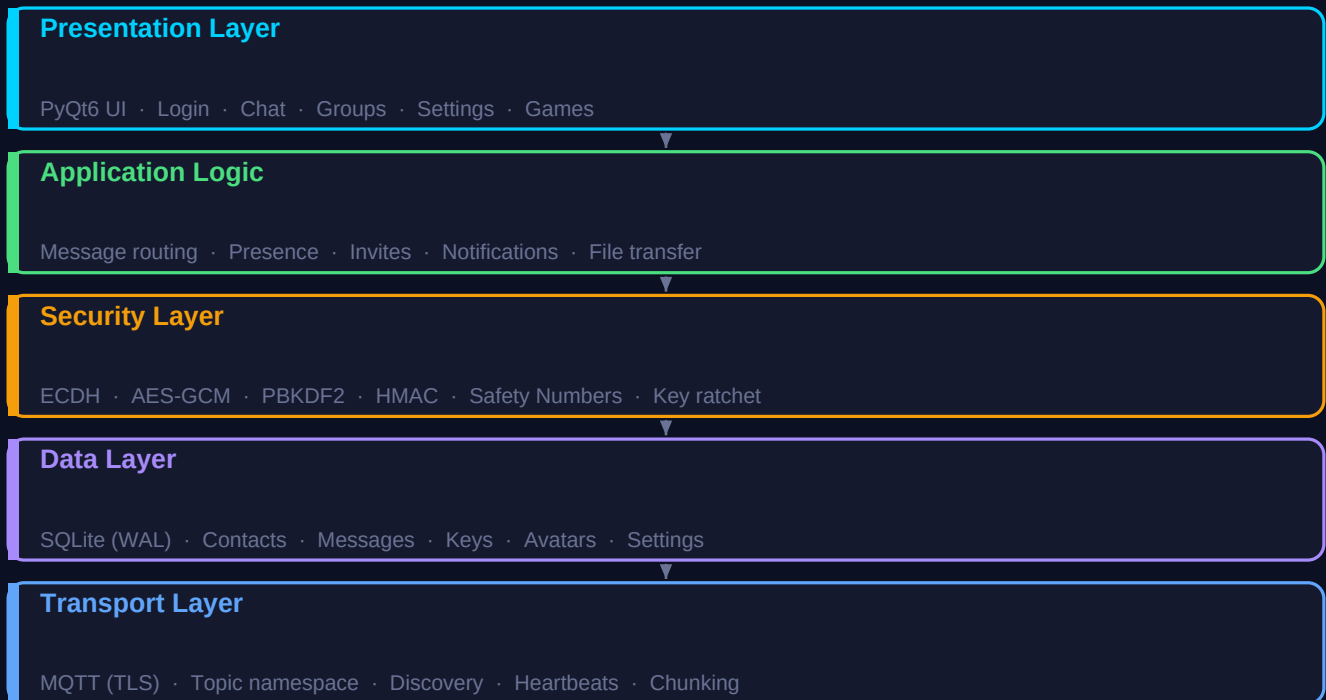
Open algorithms, no security theater

Modern UX

Dark glass UI, clear hierarchy

System Architecture

KILID is a pure client application. There is no proprietary backend that stores messages. Devices discover each other and exchange encrypted payloads through MQTT topics. All cryptographic material lives in a local SQLite database protected by the user password.



High-Level Data Flow



Note: The MQTT broker only sees ciphertext and topic metadata. It never receives private keys or plaintext message content.

Cryptographic Design

Security is the foundation of KILID. Keys are generated and stored locally. Passwords never leave the device in cleartext. Session keys can rotate after a configurable number of messages or time window.

Identity Keys

Elliptic Curve SECP384R1 key pair. Private key encrypted at rest with a key derived from the user password (PBKDF2-HMAC-SHA512, 600 000 iterations).

Key Agreement

ECDH shared secret between peers. Derived session material via HKDF. Supports TOFU (Trust On First Use) with optional Safety Number verification.

Message Encryption

AES-256-GCM authenticated encryption. Nonces are unique; timestamp windows and rate limits reduce replay and flood risk.

Password & Recovery

Login password is hashed with PBKDF2. A 32-digit Recovery Safety Number encrypts a recovery blob of the identity key for offline password reset.

Group Keys

Shared group key distributed to members. Join can be gated by PIN (PBKDF2-derived AES-GCM blob) or invite code with expiry and max uses.

Transport Safety

TLS recommended for MQTT. Discovery payloads wrapped with HMAC. Hashed peer addresses prevent trivial correlation of user IDs on public brokers.

Threat Model Highlights

Protected

Message content, private keys, password, group secrets — against network observers and untrusted brokers.

Not protected by design

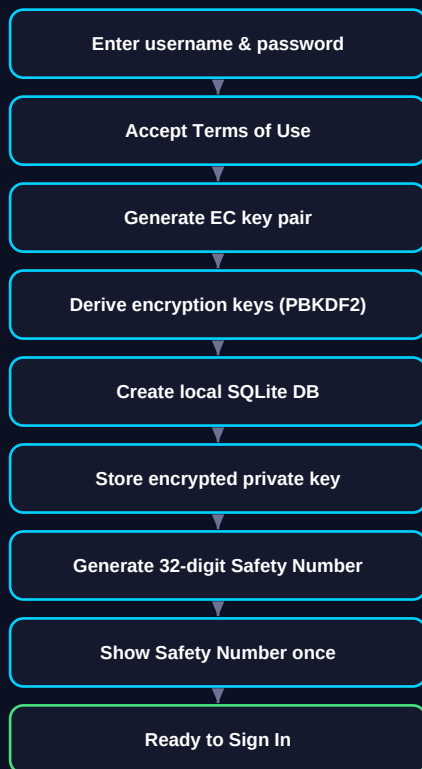
Metadata visible to the broker (who talks to whom, when, approximate size). Compromised endpoints can always read plaintext.

User responsibility

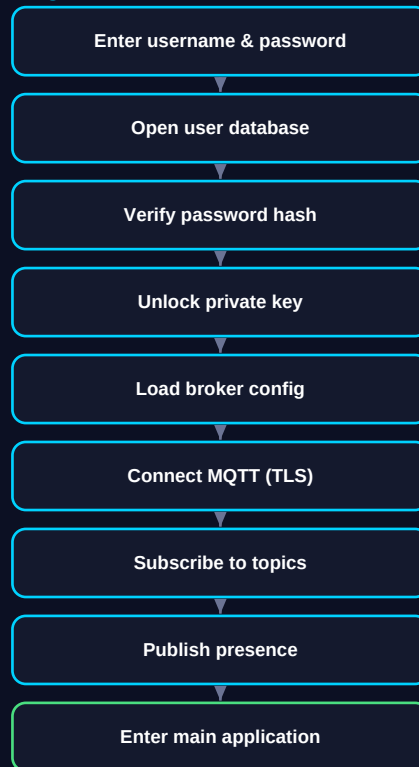
Verify Safety Numbers for high-risk contacts. Keep Recovery Safety Number offline. Use strong unique passwords.

Registration & Login Flow

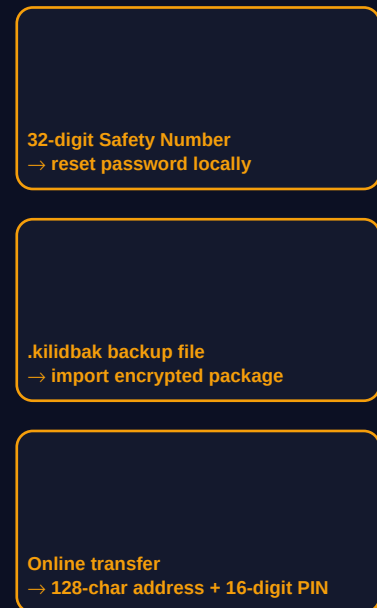
Registration



Login



Recovery Options



Failed login attempts are rate-limited (5 max). After lockout, only Safety Number recovery can reset the password.

1:1 Secure Message Path



Supported Message Types

Text Standard encrypted chat	Voice Up to 60s audio, zlib chunks	File / Image Chunked, max 10 MB
Buzz Attention ping with cooldown	Reaction Emoji reactions on messages	Group Invite Encrypted join cards
Read / Delivery Control receipts (HMAC)	Typing Ephemeral typing indicator	Profile Request / response of avatar & bio
Game Chess / Backgammon state sync	Delete / Edit Remote delete & edit markers	System Local system notices

Rate Limits & Hardening

Per-sender: 60 messages / 10 s window · Global: 300 / window · Key-exchange: 5 attempts / minute per peer · Timestamp window ± 3 min · Max encrypted packet 512 KB · Ratchet every 50 messages or 1 hour.

Main Application Views

The interface uses a consistent dark glass theme: deep navy backgrounds, cyan accents, soft borders, and clear typography. Navigation is sidebar-driven; chat is the primary surface.

Login / Register

Frameless glass dialog. Sign in, create account (with Terms acceptance), or restore via Safety Number, backup file, or online transfer. Loading overlay and attempt lockout.

Conversation List

Sidebar of contacts and groups with unread badges, last-message preview (metadata only at list level), pin / mute, and search. Avatars and online status indicators.

Chat View

Message bubbles (sent / received), timestamps, delivery & read ticks, reactions, reply, voice player, file cards, buzz animation. Composer with attach, voice, and emoji.

Groups

Create / join groups, admin tools, invite codes, PIN-protected join, member list, shared group avatar and description. Encrypted group message fan-out.

Contacts & Profile

Add by node ID / alias, notes, block list, profile bio and avatar (center-cropped 512×512 JPEG). Safety Number display for verification.

Settings

Broker configuration (presets + custom TLS), theme, notifications, password change (rotates Safety Number), export backup, account transfer, about.

Games Hub

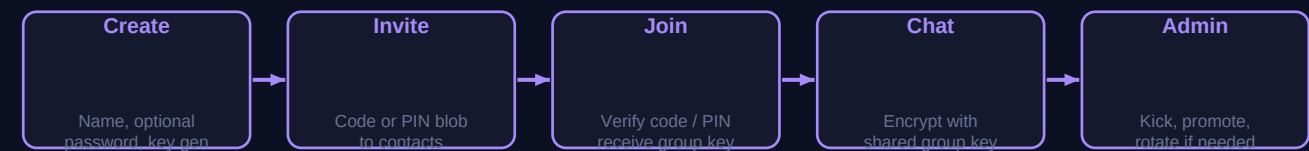
Backgammon and Chess: vs Bot, Practice, or Online challenge. Seeking mode broadcasts availability to contacts. Full board rendering and turn sync.

System Tray

Minimize to tray, notification popups for new messages and invites, quick reconnect status.

Encrypted Groups

Groups use a shared symmetric key known only to members. Creators can set a password (PBKDF2 hash), generate invite codes with expiry and max uses, and promote admins. Join payloads can be wrapped in a PIN-derived AES-GCM blob for secure out-of-band sharing.



Built-in Games

KILID includes fully playable Backgammon and Chess boards. Matches can be local (vs bot or practice) or online against a contact. Game state is sent as encrypted MQTT payloads of type "game", so play stays private.



Online Match Flow

Seeking toggle broadcasts availability → contact challenges → invite / accept → color assignment → moves & rolls exchanged as encrypted game packets → resign or win.

Technology Stack

Language	Python 3	UI Framework	PyQt6 + qdarkstyle + qtawesome
Transport	paho-mqtt (TLS via certifi)	Crypto	cryptography (ECDH, AES-GCM, PBKDF2, HMAC)
Database	SQLite (WAL mode, connection pool)	Audio	PyAudio + wave (optional)
QR / Icons	pyqrcode, qtawesome	Platform	Windows, Linux, macOS

Local Data Layout

All user data lives under ~/.kilid/ — data/<username>.db for the SQLite database, logs/ for rotating application logs. Databases are created per username; peers appear as placeholder rows until keys are established (TOFU).

Broker Defaults & Presets

HiveMQ broker.hivemq.com : 8883 TLS	Mosquitto test.mosquitto.org : 8883 TLS	EMQX broker.emqx.io : 8883 TLS	Custom User-defined host / port / TLS / auth
---	---	--	--

Updates

The app can check per-OS manifest JSON files (Windows / Linux / macOS) hosted on GitHub for version, MD5, and download URL. Users stay in control of when to apply updates.

Why KILID?

True E2E

Content is encrypted before it reaches any network. Brokers only see ciphertext.

No lock-in

Public or private MQTT brokers; open crypto; local SQLite you can inspect.

Complete product

Chat, groups, voice, files, recovery, settings, and games in one desktop app.

Thoughtful UX

Dark glass design, clear hierarchy, Safety Numbers, and recovery paths that work offline.

Responsible defaults

TLS preferred, rate limits, timestamp windows, key rotation, and strict Terms of Use.

Quick Start

1. Install dependencies (PyQt6, cryptography, paho-mqtt, ...) and run the application.
2. Create an account — save the 32-digit Recovery Safety Number offline.
3. Optionally configure a preferred MQTT broker under Settings (TLS recommended).
4. Share your node address / Safety Number with contacts and start encrypted chats.
5. Explore groups, voice messages, and online Backgammon or Chess with friends.



KILID Messenger

Secure. Local-first. Open.

<https://kilid.app> · github.com/armanghobadi/kilid

v1.0.0 · Product Introduction · 2026